

AIR Diagnostics

Within AIR we have been adding a number of low-impact diagnostic mechanisms to try to help developers get information about AIR runtime failures, from real-world scenarios i.e. when end users are using their applications. The mechanism we have created requires a developer to opt in to start receiving information, and they will need to process this or pass it off to a server for further analysis.

Some information may be provided as it happens – for example as events that the application can subscribe to – but there is also the expectation that some information would be logged to storage which can then be retrieved following an unexpected shutdown of the app (i.e. if it crashes or is aborted by an Android ANR error).

To enable diagnostics, an ANE (com.harman.air.AIRDiagnostics) will need to be included in the application. This is platform-agnostic and its inclusion will enable the diagnostic system, which can then be configured according to the particular needs. Initially there are only a few areas that are being updated to include diagnostics, but this will increase as required when customers have needs or where particular issues are suspected of causing problems. In all cases, the impact of diagnostic calls can be limited by only enabling the categories that are required for a particular application.

Below is a description of the main ANE class, as well as the utility classes that are also included in the ANE package. Following this is a worked example of an application that sets up the diagnostics and starts monitoring various aspects of the runtime, then providing information that can be presented to the user or sent as a report to a web service for later analysis.

com.harman.air.AIRDiagnostics

This is the main class for the ANE which creates and calls the ExtensionContext object to control the native diagnostic functionality. For fine control over the functionality, use this class, although it may be simpler to use the “DiagnosticInfo” class mentioned below.

Member	Description
cleanOnExit : Boolean	Read-write property that determines whether or not the diagnostic logs are cleaned up when the application is gracefully shut down. The default value is true. It is not a good idea to leave diagnostic logs un-cleaned, because these can mount up over time (but they could be cleaned up separately if required).
configure(nBitfield : uint) : Boolean	Function to configure the ANE – this needs to be called before the diagnostics system will start recording. The bitfield is how you determine which categories of diagnostic information will be captured, it should be a bitwise ‘OR’ of the values listed below (and which will be extended as additional categories are added). Note that this function should return “true” on success.
dispose() : void	Switches off and cleans up the diagnostic mechanism.
diagFolder : File	Read-only property that provides the folder in which all diagnostic reports will be added (within subfolders).
activeIndex : uint	Read-only property giving the current ‘index’ for the active diagnostic session. This is a uint based on the start-up time, and is used as the name of a subfolder under the “diagFolder” location into which the session’s log files are added.
logFolders : Vector.<File>	Read-only list of the diagnostic folders that exist. This can be used at start-up, prior to configuration, to find if there were any log folders that had not been cleaned up from the previous session(s) which may then contain useful information about crashes or ANRs.

The expected usage is to look for any “logFolders” that exist upon start-up, and if there are some, the logs for these can be examined for any useful information. Then the diagnostics can be configured for the new session. If the application is closed via a crash or ANR, then it will likely leave the “activeIndex” logs still present that can be detected on the subsequent start-up.

During an active session, some events may also be dispatched. From the ANE class these will be “StatusEvent” objects, with the ‘code’ and ‘level’ properties as follows:

Code	Level
Error	Description of the error condition that has just occurred within the AIR runtime.
LongFunction	For any function that completes having taken over two seconds, this event will provide the name of the function that has taken up the time, plus the time value (added after a tab character).

Additional event types will be added as necessary.

com.harman.air.DiagnosticInfo

This class is the main utility that should be used for accessing the ANE class and functionality. So rather than creating an “AIRDiagnostics” object, you can instead use this class which adds some additional logic and processing in order to provide a better interface. It is a singleton object and has ‘start’ and ‘stop’ methods to control the running of the underlying diagnostic mechanism. It then wraps up the log files and reports, and the status events, via additional utility classes described later.

Member	Description
instance : DiagnosticInfo	Read-only static property to get the singleton instance of this class. Use this rather than the normal constructor (on first call, this creates the singleton value which is then used in future calls).
start(filter : uint = 0xffffffff) : void	Function to start diagnostics, using a filter that is defined in the same way as the AIRDiagnostics.configure() method, see below. Note that the default value is to log everything.
stop() : void	Closes down the active diagnostics. They can be started again if required.
reports : Vector.<DiagnosticReport>	Read-only property to return a list of the diagnostic report objects that have been created from any previous diagnostic sessions.
clearReports() : void	Method to clean up any previous diagnostic session logs. This can be used following the above method i.e. get any earlier logs and then clean up.
getLog(file : String) : Vector.<String>	Gets the log for a particular category of diagnostics. Each of the configured diagnostic categories has an associated filename that should be passed in here, and if a log exists in the current session, this information is provided back to the caller as a list of log entries.
getLogFile(file : String) : File	Gets the actual log file for a particular category of diagnostics. This allows the caller to then process the file themselves, upload it to a server, etc.

When events are dispatched from the ANE, these are interpreted by this class and dispatched using the DiagnosticEvent objects.

com.harman.air.DiagnosticEvent

The events contain the information from the below table, and are dispatched from the DiagnosticInfo class above. They have read-only properties for “description” (a String) and “metric” (a Number).

There are two types:

- DiagnosticEvent.ERROR_DETAILS = “errorDetails”
 - The “description” property contains details of the error.
- DiagnosticEvent.LONG_RUNNING_FUNCTION = “longRunningFunction”
 - The “description” property contains the function that took a while to complete.
 - The “metric” property provides the duration of the function (in milliseconds).

com.harman.air.DiagnosticReport

The DiagnosticReport class is created from within the DiagnosticInfo class based on a previous diagnostic session. Currently the logs are provided just as a combined string containing all of the different categories, but this may be refined as required in the future.

Member	Description
DiagnosticReport (folder : File)	Constructor that takes the diagnostic subfolder in which the log files can be found, for this session report. The constructor reads all the categories and log files that had been recorded, and makes them available via the ‘toString’ method.
toString() : String	Provides a string that contains the full report with all log files concatenated together with each log line prefixed by the name of the category.
readLog(f : File) : Vector.<String>	Static method that allows a specific file to be passed in (the actual file generated on the system for a particular category), returning the log as a vector of String log lines. This is a method that is used by the DiagnosticInfo.getLog() method, but could also be used directly to obtain a subset of earlier log information.
getLogFile(file : String) : File	Gets the actual log file for a particular category of diagnostics from this report. This allows the caller to then process the file themselves, upload it to a server, etc. Returns null if there is no diagnostic file for this category.

Diagnostic Categories

The active categories of diagnostics are listed below, with the bitfield value as well as the name of the log file that would be created for this category (i.e. the value to use in DiagnosticInfo.getLog()).

Category identifier	Value	Log name	Description
DiagReportLog	0x0001	N/A	Detailed logging and traces – not yet implemented, please ignore.
DiagCheckLongFunc	0x0002	“long_funcs”	Log of function calls that are taking a long time (entries are added for every second that a function has been running).
DiagLogAbortOOM	0x0004	N/A	Information when an out-of-memory situation has occurred – not yet implemented, please ignore.
DiagLogMemAllocs	0x0008	N/A	Log of all operating system calls for memory allocations and frees – not yet implemented, please ignore.
DiagLogJITLocs	0x0010	N/A	Function pointer information for JIT generated functions – not yet implemented, please ignore.
DiagLogGC	0x0020	“gc_signals”	Log of state transitions within the garbage collector so that the current status and time taken in each phase can be retrieved.
DiagLogGPUDecode	0x0040	N/A	Information about decoding of data via the GPU – not yet implemented, please ignore.
DiagLogErrorCallstacks	0x0080	“error”	Error information for when an unexpected problem has been found within the AIR runtime.
DiagLogAudio	0x0100	“audio”	Information about audio decoding and playback (note – this log may get very large very quickly).
DiagLogGraphics	0x0200	“graphics”	Information about the rendering mode and graphical/display set-up within the AIR windows.
DiagLogWorker	0x0400	“worker”	Information about the Worker and MessageChannel states and changes.

Sample Code

The below code snippets show how an application might use the diagnostics mechanism. These functions would typically be called during an application start-up.

```
private function checkDiagnosticsFromLastSession() : void
{
    var diagInfo : DiagnosticInfo = DiagnosticInfo.instance;
    if (diagInfo)
    {
        // check if we have any old reports
        var reports : Vector.<DiagnosticReport> = diagInfo.reports;
        if (reports.length)
        {
            // deal with them
            trace("We have " + reports.length + " diagnostic reports");
            for each (var report : DiagnosticReport in reports)
            {
                // TODO: send data to a server for analysis
                trace(report.toString());
            }
        }
        // clear up
        diagInfo.clearReports();
    }
}
```

```

private function setupDiagnosticLogging() : void
{
    var diagInfo : DiagnosticInfo = DiagnosticInfo.instance;
    if (diagInfo)
    {
        // listen out for concerns during the runtime
        diagInfo.addEventListener(DiagnosticEvent.LONG_RUNNING_FUNCTION, function(e:DiagnosticEvent) : void {
            // TODO: capture this information or send it for processing?
            // (If it caused an ANR, it would be provided as part of the log file to be discovered on next start-up)
            trace("LONG-RUNNING FUNCTION: " + e.description + " = " + e.metric);
        });
        diagInfo.addEventListener(DiagnosticEvent.ERROR_DETAILS, function(e:DiagnosticEvent) : void {
            trace("DIAGNOSTIC ERROR: " + e.description + " = " + e.metric);
            // TODO: handle specific issues for example if a MessageChannel "ChannelClosed" error has happened:
            if (e.description.startsWith("Throwing ChannelClosed error"))
            {
                trace(" -- checking for worker log --");
                var log : Vector.<String> = diagInfo.getLog("worker");
                // TODO: send to diagnostics server
                if (log) for (var i : uint = 0; i < log.length; i++) trace(log[i]);
            }
        });
        // start the diagnostics, pick up long functions, GC events, and errors
        diagInfo.start(AIRDiagnostics.DiagCheckLongFunc |
                      AIRDiagnostics.DiagLogGC          |
                      AIRDiagnostics.DiagLogErrorCallstacks);
    }
}

```